Weekly Work Report

DORA Ranking Portal – Organogram-Driven Data Collection & Nodal Officer Access

Contributor: KDS-7Git (kdshacker359@gmail.com)
Period: 19 August 2026 – 9 September 2026
Repository: https://github.com/rishang19dx/dora-ranking-portal
Source commit: a52adde – *feat: implement access management for nodal officers*

Basis of this report

- **Scope.** Only work authored by KDS-7Git is covered. Commits by other contributors (UI overhaul, role-based routing, nodal-officer upload UI, cache webhook, and the original NIRF/organogram baseline by a teammate) are excluded.
- **Commit history.** All work was done locally and committed once: a single commit, a52adde, dated 9 Sep 2026, 23:57 IST (32 files, +5,926 / −173 lines). The repository therefore contains no per-week commits.
- **Line counts** are the final size of each file in commit a52adde. Pages built in Week 1–2 were later edited in Week 3 to wire in the organogram.

Summary

Week	Dates & focus	Final lines (files built)	Effort (h)
1	19–25 Aug: server workflow, access, persistence, API	1,837	8
2	26 Aug–1 Sep: admin UI, upload portal, validation, docs	2,502	7
3	2–9 Sep: organogram model & integration	1,587	9
	Total	**5,926**	**24**

Line counts are exact (from `git show -numstat a52adde`). Week 3 effort includes rework of Week 1–2 files for the organogram, which is not counted in the line column.

Week 1 (19–25 Aug): Server workflow, access and persistence

Effort:8 hours

- **Persistence.** src/lib/server/store.ts (283 lines): single file-backed repository for all workflow state. Mirrored Prisma models in prisma/schema.prisma (203 lines) so moving to Postgres touches one file.
- **Access provisioning.** access.ts (110 lines): hashed, expiring, single-section upload grants.
- **Dispatch.** dispatch.ts (339 lines): targeted requests, one grant per recipient, link rotation on reminder, revocation on close or officer removal.
- **Mailer, coverage, HTTP helpers.** mailer.ts (128), coverage.ts (36), http.ts (47).
- **Shared types.** src/lib/types.ts (118 lines).
- **API routes.** api/data-requests (193 lines), api/nodal-officers (230 lines), api/portal/[token]/upload (125 lines): uploads scoped to the grant and checksummed server-side.
- **Configuration.** .env.example (20), .gitignore (5).

Key decision: all state sits behind one repository module.

Week 2 (26 Aug–1 Sep): Admin UI, upload portal, validation, documentation

Effort: 7 hours

- **Request composer.** `admin/data-requests/new/page.tsx` (730 lines): three steps (sections → required data → message), live recipient resolution, coverage warnings, templated email preview.
- **Request detail.** `admin/data-requests/[id]/page.tsx` (465 lines): status roll-up, reminders, close.
- **Nodal officer management.** `admin/nodal-officers/page.tsx` (+664/−117) and `admin/data-collection/page.tsx` (+249/−52); small `DashboardLayout.tsx` change.
- **Restricted upload portal.** `portal/[token]/page.tsx` (124) and `PortalUploader.tsx` (191).
- **Validation.** `npm run build` passes. Flow exercised end to end on a dev server: create officer → dispatch → outbox email → portal upload → status roll-up → reminder rotation → close/revoke, including rejection of wrong-format, out-of-scope, revoked and invalid-token uploads.
- **Documentation.** `README.md`, `decision_history.md`.

Week 3 (2–9 Sep): Organogram integration

Effort: 9 hours

- **Organogram modelling.** Converted the teammate's `Institute_Organogram.md` into a typed, queryable hierarchy in `src/lib/organogram.ts` (577 lines): 56 units across governance, executive, academic, administrative and functional layers, with per-unit data custodianship.
- **Data catalogue.** `data-catalogue.ts` (320 lines): 28 requestable items grouped by NIRF parameter and mapped to the units that hold them.
- **Reusable tree.** `OrgTree.tsx` (300 lines): searchable, selectable hierarchy; `admin/organogram/page.tsx` (359) and `api/organogram/route.ts` (31).
- **Integration.** Reworked the composer, officer roster, dispatch and coverage logic to use the organogram (per-section narrowing of requested items, coverage warnings, roster seeded from organogram mailboxes), then re-ran the end-to-end flow and build.

Key decision: the hierarchy lives in code, so one unit→data mapping drives section selection and per-section narrowing.

Technical decisions logged

1. **Organogram modelling:** typed hierarchy with per-unit data custodianship; reusable searchable tree.
2. **Targeted requests:** three-step composer with per-section narrowing and coverage warnings.
3. **Access provisioning:** one hashed, expiring, single-section grant per recipient; rotation on reminder; revocation on close or removal; scoped, checksummed uploads.
4. **Persistence:** single file-backed repository module with mirrored Prisma models.
5. **Validation:** build and end-to-end flow exercised, including negative upload cases.